Session 11B Teamwork — Close the Same Gap on Workspaces, and Prove It

Context

Work in your team (six teams of four students). CORE just gave Projects real improvements: diagnosed and fixed a real dependency-injection failure, closed the last `ParseIntPipe` gap, and migrated Projects' data to real, persisted MongoDB documents through Mongoose — verified independently through `curl`, `mongosh`, and MongoDB Compass, including real `?name=...` / `?key=...` filtering run directly against that data. `WorkspacesController` / `WorkspacesService` have none of this yet — your job is to close every gap at the source, the same way CORE just did for Projects, and prove it yourselves in a real database.

Goal

`WorkspacesController` / `WorkspacesService` reach the same end state as Projects' own files — reasoned through by your team, not copied and renamed from `projects/*.ts` — and your team independently verifies that the data really is persisted, in your own `mongosh` session.

What to build

- Add `ParseIntPipe` to BOTH `@Param('id')` usages in `WorkspacesController` (`findOne`, `update`) — neither one has it yet.
- `WorkspaceSchema` (new file, `src/workspaces/schemas/workspace.schema.ts`): the identical shape as `ProjectSchema`, with `slug` playing the role `key` plays for Projects.
- Register the schema with `WorkspacesModule` via `MongooseModule.forFeature([...])` — the exact registration that was missing on `ProjectsModule` this morning.
- Migrate `WorkspacesService` from its in-memory array to a real, `@InjectModel`-injected Mongoose model — including the same `OnModuleInit` seed-once pattern CORE just used.
- `FindWorkspacesQueryDto` (new file, `src/workspaces/dto/find-workspaces-query.dto.ts`): the identical shape as `FindProjectsQueryDto`, with `slug` instead of `key`.
- Wire `@Query()` into `WorkspacesController`'s own `findAll()`, and the real filtering logic into `WorkspacesService`'s own `findAll()` — the identical two-step pattern CORE just used.
- **Required deliverable:** create a real workspace through `curl`, then independently confirm it exists in your own `mongosh` session, querying the real `workspaces` collection directly. Write down (or screenshot) both results side by side.

Rules

Allowed: everything the instructor just used for Projects — `ParseIntPipe`, `@Schema()` / `@Prop()` / `SchemaFactory.createForClass()`, `MongooseModule.forFeature`, `@InjectModel`, `OnModuleInit`, `@Query()`, the `@IsOptional()` / `@IsString()` combination, `mongosh` queries, exactly as shown in class (no new decorators, no new packages). **Not yet:** no pagination, no sorting, no guards, no database-level uniqueness constraints.

Test it

BASH

```
curl -i -X GET localhost:3000/workspaces/abc
curl -X GET localhost:3000/workspaces/1
curl -X GET localhost:3000/workspaces
curl -X GET "localhost:3000/workspaces?name=nova"
curl -X GET "localhost:3000/workspaces?slug=acme-robotics"
curl -i -X GET "localhost:3000/workspaces?foo=bar"
curl -X POST localhost:3000/workspaces -H "Content-Type: application/json" -d
'{"name":"Growth","slug":"growth"}'
mongosh --eval 'db.getSiblingDB("b4f-board").workspaces.find({ slug: "growth" })'
```

Done when

- `GET /workspaces/abc` returns a clean `400`, not the old confusing `404`.
- `GET /workspaces/1` still works exactly as before.
- `GET /workspaces` with no query still returns everything, now read from MongoDB.
- `GET /workspaces?name=nova` returns only the matching workspace.
- `GET /workspaces?slug=acme-robotics` returns only the matching workspace.
- `GET /workspaces?foo=bar` returns a clean `400` — the unknown query key is rejected, not silently ignored.
- Your team's own `mongosh` session independently shows the exact workspace you just created through `curl` — in the real `workspaces` collection of the real `b4f-board` database.
- Every team member can explain why the controller change alone doesn't compile until the service is updated too — twice: once for `@InjectModel`, once for `@Query()`.

Working time

About 50 minutes — the identical full pattern you already watched built once on Projects, applied independently to a second resource, plus a real database check you run yourselves.